Assignment 1_DATA 608

Mehreen Ali Gillani

2024-06-10

Story - 1 : Infrastructure Investment & Jobs Act Funding Allocation

Import Libraries and Load Data

```
#import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
url = "https://github.com/mehreengillani/DATA608/blob/main/IIJA%20FUNDING%20AS%20OF%20MARCH%2
data = pd.read_excel(url)
# Display the first few rows of the dataset
data.head()
```

	State, Teritory or Tribal Nation	Total (Billions)
0	ALABAMA	3.0000
1	ALASKA	3.7000
2	AMERICAN SAMOA	0.0686
3	ARIZONA	3.5000
4	ARKANSAS	2.8000

Data Overview

```
# First, let's see what's in your data
print("DataFrame 'data' overview:")
print(f"Shape: {data.shape}")
print(f"Columns: {data.columns.tolist()}")
print("\nFirst few rows:")
print(data.head())
```

```
DataFrame 'data' overview:
Shape: (57, 2)
Columns: ['State, Teritory or Tribal Nation', 'Total (Billions)']

First few rows:
  State, Teritory or Tribal Nation  Total (Billions)
0                          ALABAMA            3.0000
1                           ALASKA            3.7000
2                   AMERICAN SAMOA            0.0686
3                          ARIZONA            3.5000
4                         ARKANSAS            2.8000
```

Check for missing values

```
# Check for missing values
print("\nMissing values in each column:")
print(data.isnull().sum())
```

```
Missing values in each column:
State, Teritory or Tribal Nation    0
Total (Billions)                    0
dtype: int64
```

Load Election Data

```
# Get the voting data
#https://www.kaggle.com/datasets/callummacpherson14/2020-us-presidential-election-results-by-
#saved it to github for easy access
url = "https://raw.githubusercontent.com/mehreengillani/DATA608/refs/heads/main/voting.csv"
```

```python
election_data = pd.read_csv(url, delimiter=',', quoting=3)
election_data.head()
print("VOTING DATA:")
print(f"Shape: {election_data.shape}")
print(f"Columns: {election_data.columns.tolist()}")
print("\nFirst 5 rows:")
print(election_data.head())

# Clean voting data - ensure state names are consistent
election_data['state_clean'] = election_data['state'].str.upper()
print(f"\nUnique states in voting data: {len(election_data['state_clean'].unique())}")
```

```
VOTING DATA:
Shape: (51, 8)
Columns: ['state', 'state_abr', 'trump_pct', 'biden_pct', 'trump_vote', 'biden_vote', 'trump_

First 5 rows:
        state state_abr  trump_pct  biden_pct  trump_vote  biden_vote  \
0      Alaska        AK       53.1       43.0      189543      153502
1      Hawaii        HI       34.3       63.7      196864      366130
2  Washington        WA       39.0       58.4     1584651     2369612
3      Oregon        OR       40.7       56.9      958448     1340383
4  California        CA       34.3       63.5     5982194    11082293

   trump_win  biden_win
0          1          0
1          0          1
2          0          1
3          0          1
4          0          1

Unique states in voting data: 51
```

Load Population Data

```python
# Population data -
url1="https://raw.githubusercontent.com/mehreengillani/DATA608/refs/heads/main/NST-EST2025-P
# Merge funding data with election data
# Read population data
population_data = pd.read_excel(url1, header=None, skiprows=3)
```

```python
# Clean population data - extract states (rows 5-55 are states, based on your data)
states_data = population_data.iloc[5:57].copy()  # Rows 5-57 inclusive

# Assign proper column names
states_data.columns = ['state', '2020_base', '2020', '2021', '2022', '2023', '2024', '2025']

# Clean state names - remove leading dots and trim
states_data['state_clean'] = (
    states_data['state']
    .astype(str)
    .str.replace('^\.+', '', regex=True)  # Remove leading dots
    .str.strip()  # Remove whitespace
    .str.upper()  # Convert to uppercase
)

# Convert population columns from strings with commas to numbers
for col in ['2020_base', '2020', '2021', '2022', '2023', '2024', '2025']:
    states_data[col] = (
        states_data[col]
        .astype(str)
        .str.replace(',', '', regex=True)  # Remove commas
        .str.replace(' ', '', regex=True)  # Remove spaces
        .replace('nan', np.nan)  # Convert 'nan' string to actual NaN
    )
    states_data[col] = pd.to_numeric(states_data[col], errors='coerce')

# Select 2020 population
states_data['population'] = states_data['2020']

# Create clean population dataframe
population_clean = states_data[['state_clean', 'population']].dropna()

print(f"Population data: {len(population_clean)} states")
print("\nFirst 5 states:")
print(population_clean.head())

print("\nLast 5 states:")
print(population_clean.tail())
```

```
Population data: 52 states
```

```
First 5 states:
  state_clean  population
5        WEST  78689140.0
6     ALABAMA   5032962.0
7      ALASKA    732906.0
8     ARIZONA   7186647.0
9    ARKANSAS   3014399.0

Last 5 states:
      state_clean  population
52       VIRGINIA   8637303.0
53     WASHINGTON   7726812.0
54  WEST VIRGINIA   1791670.0
55      WISCONSIN   5897371.0
56        WYOMING    577669.0
```

Merge datasets and check for issues

```python
merged = pd.merge(election_data, population_clean, on='state_clean', how='left')

print(f"\nMerged: {len(merged)} rows")
print(f"Missing population: {merged['population'].isna().sum()} states")

# Show sample
print("\nSample merged data:")
print(merged[['state', 'state_abr', 'biden_pct', 'population']].head())
```

```
Merged: 51 rows
Missing population: 0 states

Sample merged data:
        state state_abr  biden_pct  population
0      Alaska        AK       43.0    732906.0
1      Hawaii        HI       63.7   1451130.0
2  Washington        WA       58.4   7726812.0
3      Oregon        OR       56.9   4243544.0
4  California        CA       63.5  39527808.0
```

```python
# Check which states are missing population
missing = merged[merged['population'].isna()]
print("States missing population data:")
print(missing[['state', 'state_abr']].to_string(index=False))

# Check state count
print(f"\nTotal states with population: {merged['population'].notna().sum()}")
print(f"Total states expected: 51 (50 states + DC)")

# Check specific problematic states
problem_states = ['DISTRICT OF COLUMBIA', 'PUERTO RICO', 'VIRGIN ISLANDS']
for state in problem_states:
    matches = merged[merged['state_clean'] == state]
    if len(matches) > 0:
        print(f"\n{state}:")
        print(f"  In merged data: {len(matches)} rows")
        print(f"  Population: {matches['population'].iloc[0] if len(matches) > 0 else 'Not f
```

```
States missing population data:
Empty DataFrame
Columns: [state, state_abr]
Index: []

Total states with population: 51
Total states expected: 51 (50 states + DC)

DISTRICT OF COLUMBIA:
  In merged data: 1 rows
  Population: 670958.0
```

Merge with infrastructure funding data and calculate metrics

```python
# Merge with infrastructure funding data
print("\n" + "="*60)
print("ADDING INFRASTRUCTURE FUNDING DATA")
print("="*60)

# Clean your infrastructure data
data['state_clean'] = data['State, Teritory or Tribal Nation'].str.upper()
```

```python
# Merge all three datasets
final_data = pd.merge(merged, data[['state_clean', 'Total (Billions)']],
                      on='state_clean',
                      how='left')

print(f"Final dataset: {len(final_data)} rows")
print(f"States with infrastructure data: {final_data['Total (Billions)'].notna().sum()}")

# Calculate key metrics
final_data['funding_per_capita'] = (final_data['Total (Billions)'] * 1e9) / final_data['popul
final_data['winner'] = np.where(final_data['biden_win'] == 1, 'Biden', 'Trump')

print("\nFinal data sample:")
sample_cols = ['state', 'state_abr', 'winner', 'population', 'Total (Billions)', 'funding_per
print(final_data[sample_cols].head(10).round(2))

print(f"\nKey statistics:")
print(f"Total funding: ${final_data['Total (Billions)'].sum():.2f}B")
print(f"Average funding per capita: ${final_data['funding_per_capita'].mean():,.0f}")
print(f"States won by Biden: {(final_data['winner'] == 'Biden').sum()}")
print(f"States won by Trump: {(final_data['winner'] == 'Trump').sum()}")
```

```
============================================================
ADDING INFRASTRUCTURE FUNDING DATA
============================================================
Final dataset: 51 rows
States with infrastructure data: 50

Final data sample:
        state state_abr winner  population  Total (Billions)  \
0      Alaska        AK  Trump    732906.0               3.7
1      Hawaii        HI  Biden   1451130.0               1.0
2  Washington        WA  Biden   7726812.0               4.0
3      Oregon        OR  Biden   4243544.0               2.3
4  California        CA  Biden  39527808.0              18.4
5       Idaho        ID  Trump   1849328.0               1.2
6     Montana        MT  Trump   1087156.0               3.3
7      Nevada        NV  Biden   3116851.0               1.7
8     Wyoming        WY  Trump    577669.0               2.3
9        Utah        UT  Trump   3283970.0               1.8
```

```
   funding_per_capita
0             5048.40
1              689.12
2              517.68
3              542.00
4              465.50
5              648.88
6             3035.44
7              545.42
8             3981.52
9              548.12

Key statistics:
Total funding: $190.80B
Average funding per capita: $913
States won by Biden: 26
States won by Trump: 25
```

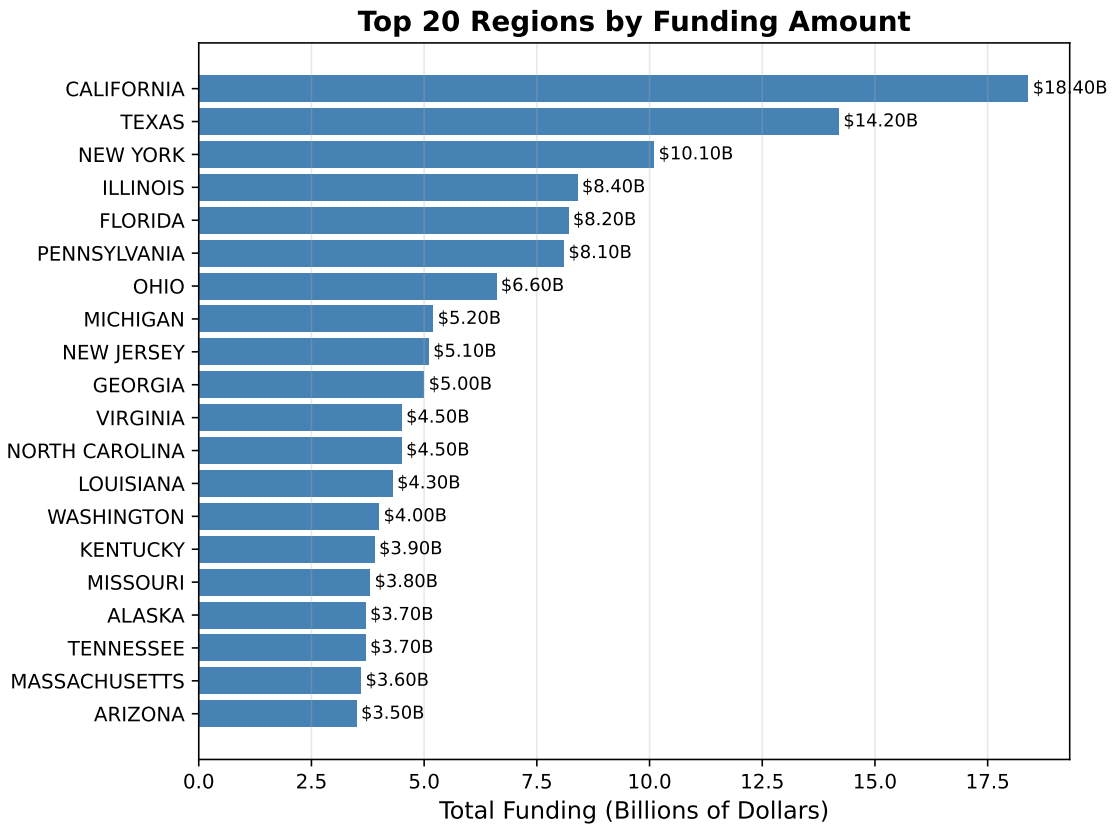
We have a clean, merged dataset with key metrics calculated. We can proceed to create visualizations that compare funding per capita across states, highlighting differences based on the 2020 election results.

Basic Visualization

```
print("\n" + "="*60)
print("STEP 2: BASIC VISUALIZATION")
print("="*60)

# Sort data for better visualization
data_sorted = data.sort_values('Total (Billions)', ascending=False)

# Create a simple bar chart
plt.figure(figsize=(8, 6))

# Only show top 20 for clarity
top_20 = data_sorted.head(20)

plt.barh(top_20['State, Teritory or Tribal Nation'], top_20['Total (Billions)'], color='steel
plt.xlabel('Total Funding (Billions of Dollars)', fontsize=12)
plt.title('Top 20 Regions by Funding Amount', fontsize=14, fontweight='bold')
```

```
plt.gca().invert_yaxis()  # Highest at top
plt.grid(axis='x', alpha=0.3)

# Add value labels on bars
for i, (value, region) in enumerate(zip(top_20['Total (Billions)'], top_20['State, Teritory o
    plt.text(value + 0.1, i, f'${value:.2f}B', va='center', fontsize=9)

plt.tight_layout()
plt.show()

print(f"Top 5 funded regions:")
for i, row in data_sorted.head(5).iterrows():
    print(f"  {row['State, Teritory or Tribal Nation']}: ${row['Total (Billions)']:.2f} billi
```

```
============================================================
STEP 2: BASIC VISUALIZATION
============================================================
```

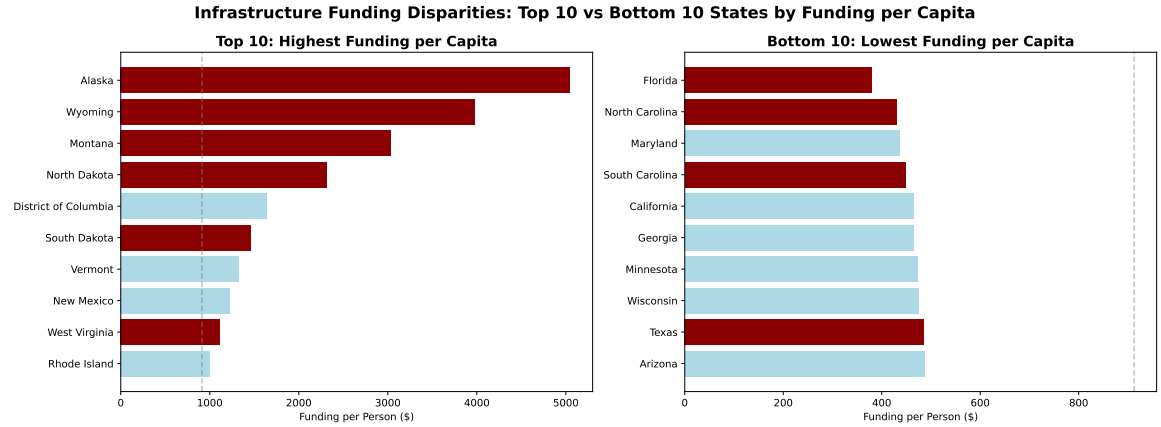



```
Top 5 funded regions:
  CALIFORNIA: $18.40 billion
  TEXAS: $14.20 billion
  NEW YORK: $10.10 billion
  ILLINOIS: $8.40 billion
  FLORIDA: $8.20 billion
```

Visualization: A side-by-side comparison of the top 10 and bottom 10 states in terms of funding per capita, with clear color coding to indicate which states were won by Biden vs Trump in the 2020 election.

```
# Get top 10 and bottom 10 states
top10 = final_data.nlargest(10, 'funding_per_capita')
bottom10 = final_data.nsmallest(10, 'funding_per_capita')
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Top 10 states
bars1 = ax1.barh(top10['state'], top10['funding_per_capita'],
                 color=['darkred' if w == 'Trump' else 'lightblue' for w in top10['winner']])
ax1.set_xlabel('Funding per Person ($)')
ax1.set_title('Top 10: Highest Funding per Capita', fontweight='bold', fontsize=14)
ax1.invert_yaxis()
ax1.axvline(x=final_data['funding_per_capita'].mean(), color='gray', linestyle='--', alpha=0

# Plot 2: Bottom 10 states
bars2 = ax2.barh(bottom10['state'], bottom10['funding_per_capita'],
                 color=['darkred' if w == 'Trump' else 'lightblue' for w in bottom10['winner']
ax2.set_xlabel('Funding per Person ($)')
ax2.set_title('Bottom 10: Lowest Funding per Capita', fontweight='bold', fontsize=14)
ax2.invert_yaxis()
ax2.axvline(x=final_data['funding_per_capita'].mean(), color='gray', linestyle='--', alpha=0

plt.suptitle('Infrastructure Funding Disparities: Top 10 vs Bottom 10 States by Funding per C
plt.tight_layout()
plt.show()
```

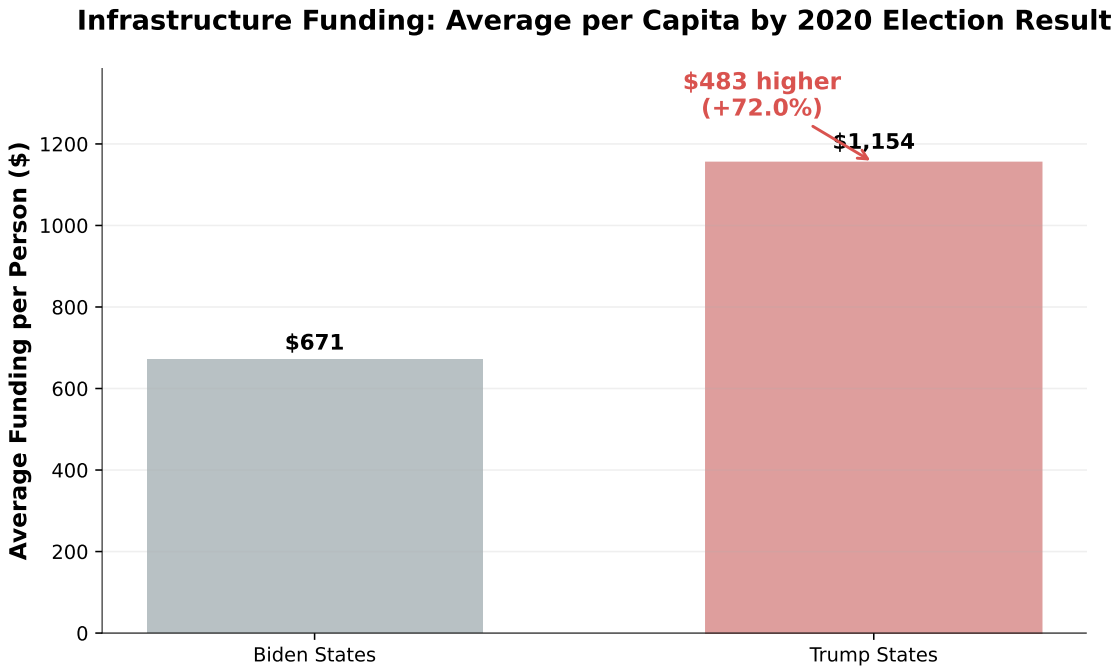


This visualization shows the stark contrast between the top 10 and bottom 10 states in terms of infrastructure funding per capita, with clear color coding to indicate which states were won by Biden vs Trump in the 2020 election. The dashed line represents the national average funding per capita for context.

Comparison of average funding per capita for Biden vs Trump states, with a clear highlight on the difference.

```
# Simple Biden vs Trump comparison
biden_avg = final_data[final_data['winner'] == 'Biden']['funding_per_capita'].mean()
trump_avg = final_data[final_data['winner'] == 'Trump']['funding_per_capita'].mean()

# Calculate difference for highlighting
diff = trump_avg - biden_avg
percent_diff = ((trump_avg/biden_avg)-1)*100

# Create figure with professional styling
fig, ax = plt.subplots(figsize=(8, 5))

# Use neutral colors with strategic highlighting
bars = ax.bar(['Biden States', 'Trump States'],
              [biden_avg, trump_avg],
              color=['#b0babe', '#95a5ad'],  # Professional slate gray for both
              alpha=0.9,
              width=0.6)

# Highlight only the Trump bar since it's higher
bars[1].set_color('#db9492')  # Professional red for highlight

# Professional styling
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_linewidth(0.5)
ax.spines['bottom'].set_linewidth(0.5)

ax.set_ylabel('Average Funding per Person ($)', fontsize=12, fontweight='semibold')
ax.set_title('Infrastructure Funding: Average per Capita by 2020 Election Result',
             fontsize=14, fontweight='bold', pad=20)

# Add clean value labels
for i, bar in enumerate(bars):
    height = bar.get_height()
    # Position labels just above bars
    ax.text(bar.get_x() + bar.get_width()/2., height + (height*0.02),
            f'${height:,.0f}',
            ha='center',
            va='bottom',
```

```
            fontweight='bold',
            fontsize=11)

# Add difference annotation
if diff > 0:
    ax.annotate(f'${diff:,.0f} higher\n(+{percent_diff:.1f}%)',
                xy=(1, trump_avg),
                xytext=(0.8, trump_avg + (trump_avg*0.1)),
                arrowprops=dict(arrowstyle='->', color='#D9534F', lw=1.5),
                fontsize=12,
                fontweight='semibold',
                color='#D9534F',
                ha='center')

# Add subtle grid
ax.grid(axis='y', alpha=0.2, linestyle='-')

# Optional: Add a subtle horizontal line at 0
ax.axhline(y=0, color='black', alpha=0.1, linewidth=0.5)

# Adjust y-axis to start at 0 with some padding
ax.set_ylim(0, max(biden_avg, trump_avg) * 1.2)

plt.tight_layout()
plt.show()

# Print statistical summary
print(f"Average funding per capita:")
print(f"  Biden states: ${biden_avg:,.0f}")
print(f"  Trump states: ${trump_avg:,.0f}")
print(f"\nTrump states receive ${diff:,.0f} more per person")
print(f"That's {percent_diff:.1f}% higher")

# Optional context
print(f"\nNote: Based on {len(final_data[final_data['winner'] == 'Biden'])} Biden states and
```



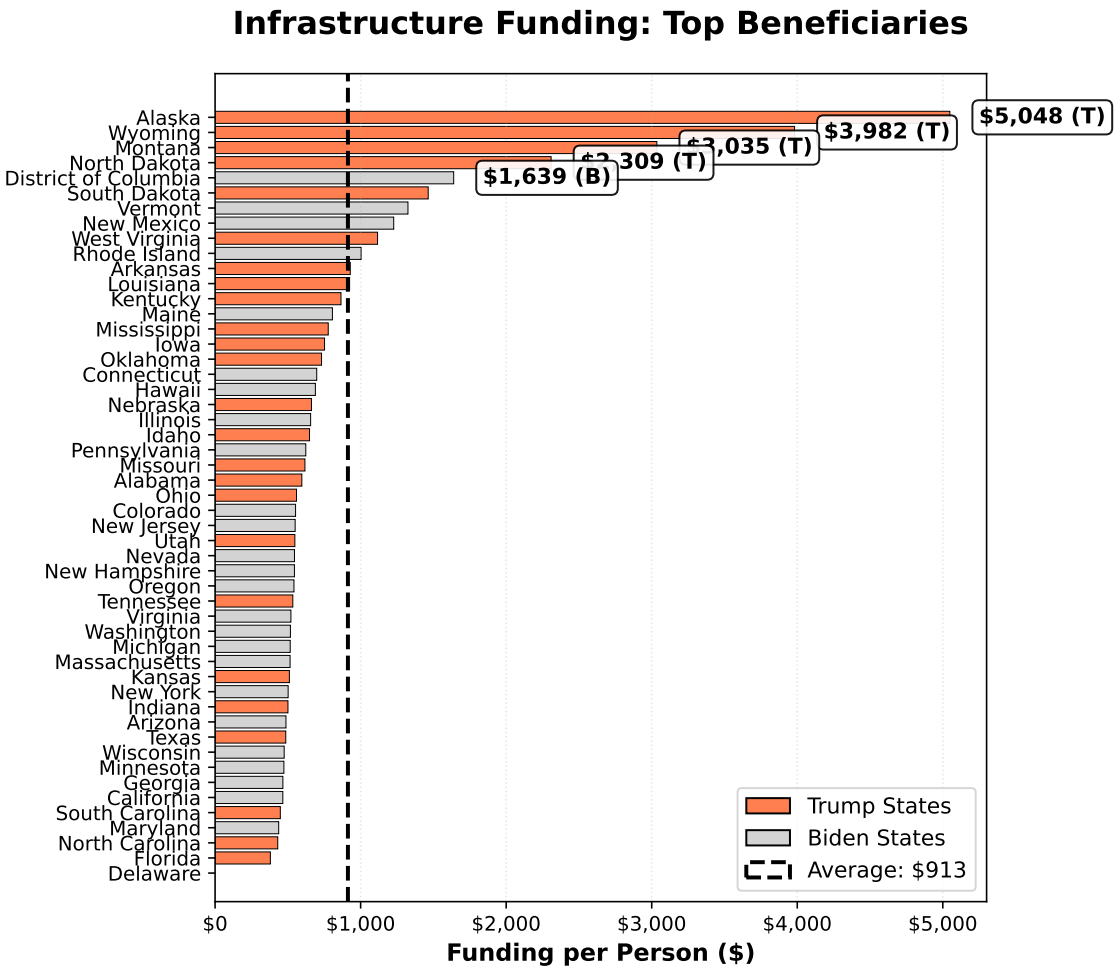

```
Average funding per capita:
  Biden states: $671
  Trump states: $1,154

Trump states receive $483 more per person
That's 72.0% higher

Note: Based on 26 Biden states and 25 Trump states
```

This visualization provides a clear comparison of average infrastructure funding per capita for states won by Biden vs Trump in the 2020 election, with a professional color scheme and annotations to highlight the difference. The statistical summary gives additional context to the findings.

```
# Sort data
final_sorted = final_data.sort_values('funding_per_capita', ascending=False)

fig, ax = plt.subplots(figsize=(8, 7))  # Increased height

# Create clean color scheme
```

```
colors = []
for _, row in final_sorted.iterrows():
    if row['winner'] == 'Trump':
        colors.append('coral')  # Trump states stand out
    else:
        colors.append('lightgray')  # Biden states subtle

bars = ax.barh(final_sorted['state'], final_sorted['funding_per_capita'],
               color=colors, edgecolor='black', linewidth=0.5)

# National average line
avg = final_data['funding_per_capita'].mean()
ax.axvline(x=avg, color='black', linestyle='--', linewidth=2,
           label=f'National Average\n${avg:,.0f} per person')

# Label only top 5 (not bottom)
top_n = 5  # Increased to 5 for better coverage

for i, (bar, row) in enumerate(zip(bars, final_sorted.iterrows())):
    row = row[1]

    # Label only top 5
    if i < top_n:
        # Position text to the right of bar with more spacing
        text_x = bar.get_width() + 200  # Increased from 100

        # Create compact label
        label_text = f'${row["funding_per_capita"]:,.0f} ({row["winner"][0]})'

        # Add label with better styling
        ax.text(text_x, bar.get_y() + bar.get_height()/2,
                label_text,
                va='center', fontsize=11, fontweight='bold',  # Increased fontsize
                bbox=dict(boxstyle='round', facecolor='white', alpha=0.9,
                          edgecolor='black', linewidth=1))

# Improve state name fonts on y-axis
ax.tick_params(axis='y', labelsize=10)  # Slightly smaller for state names

# Clean legend
from matplotlib.patches import Patch
legend_elements = [
```

```
    Patch(facecolor='coral', edgecolor='black', label='Trump States'),
    Patch(facecolor='lightgray', edgecolor='black', label='Biden States'),
    Patch(facecolor='none', edgecolor='black', linestyle='--', linewidth=2,
          label=f'Average: ${avg:,.0f}')
]
ax.legend(handles=legend_elements, loc='lower right', fontsize=11)

ax.set_xlabel('Funding per Person ($)', fontsize=12, fontweight='bold')
ax.set_title('Infrastructure Funding: Top Beneficiaries',
             fontsize=16, fontweight='bold', pad=20)  # Smaller title
ax.invert_yaxis()
ax.grid(axis='x', alpha=0.3, linestyle=':')

# Add more x-tick spacing for readability
import matplotlib.ticker as ticker
ax.xaxis.set_major_formatter(ticker.StrMethodFormatter('${x:,.0f}'))

plt.tight_layout()
plt.show()

print(f"Labeled top {top_n} states:")
for i in range(top_n):
    state = final_sorted.iloc[i]
    print(f"  {state['state']} ({state['winner']}): ${state['funding_per_capita']:,.0f}")
```




```
Labeled top 5 states:
  Alaska (Trump): $5,048
  Wyoming (Trump): $3,982
  Montana (Trump): $3,035
  North Dakota (Trump): $2,309
  District of Columbia (Biden): $1,639
```

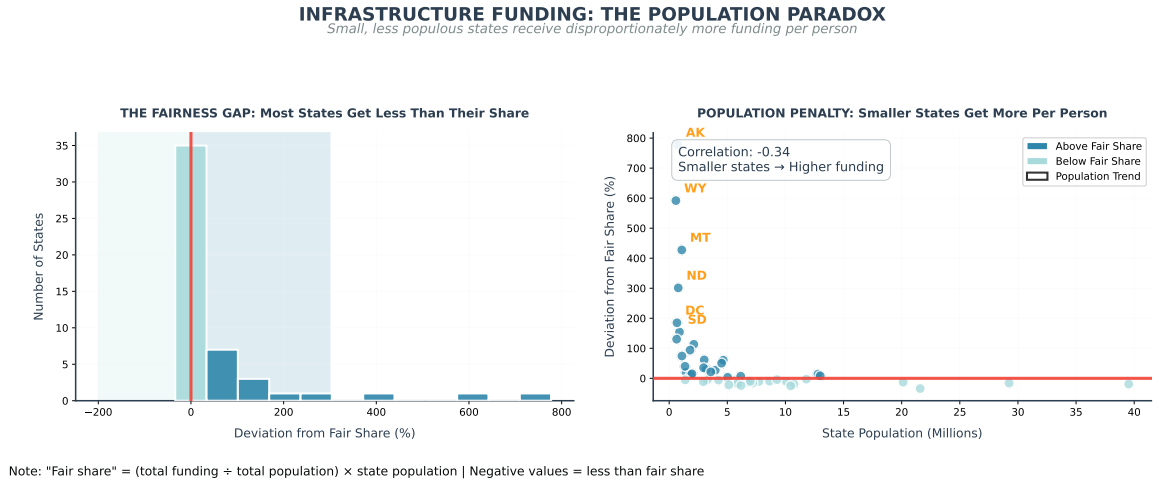
This visualization highlights the top beneficiaries of infrastructure funding per capita, with clear color coding to indicate which states were won by Biden vs Trump in the 2020 election. The dashed line represents the national average funding per capita for context, and only the top 5 states are labeled for clarity.

```
# Visualization: Fairness Analysis (without politics)
total_funding = final_data['Total (Billions)'].sum() * 1e9  # Convert to dollars
total_population = final_data['population'].sum()
fair_per_capita = total_funding / total_population

# Calculate deviation from fair
final_data['deviation'] = (final_data['funding_per_capita'] - fair_per_capita) / fair_per_ca

# Set NEW color theme focused on fairness, not politics
COLORS = {
    'above_fair': '#2E86AB',    # Blue for above fair (positive but neutral)
    'below_fair': '#A8DADC',    # Light blue for below fair
    'fair_line': '#F24236',     # Red for the fair line (0% deviation)
    'trend': '#333333',         # Dark gray for trend lines
    'grid': '#F5F5F5',          # Very light gray for grid
    'outliers': '#FF9F1C',      # Orange for highlighting outliers
    'text': '#2C3E50'           # Dark blue-gray for text
}

# Create figure with two subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# ------------------------------------------------------------------
# LEFT PLOT: Distribution of Fairness
# ------------------------------------------------------------------
deviations = final_data['deviation']

# Create histogram with custom styling
n, bins, patches = ax1.hist(deviations, bins=12, edgecolor='white', linewidth=2,
                            alpha=0.9, zorder=3)

# Color bars based on sign (above/below fair)
for i, (patch, left_edge, right_edge) in enumerate(zip(patches, bins[:-1], bins[1:])):
    midpoint = (left_edge + right_edge) / 2
    if midpoint > 0:
        patch.set_facecolor(COLORS['above_fair'])  # Blue for above fair
    else:
        patch.set_facecolor(COLORS['below_fair'])  # Light blue for below fair

# Add vertical line at 0 (fair point) in bold red
ax1.axvline(x=0, color=COLORS['fair_line'], linestyle='-', linewidth=3,
            alpha=0.9, zorder=4, label='Fair Allocation Line')
```

```
# Add background shading for regions
ax1.axvspan(-200, 0, alpha=0.15, color=COLORS['below_fair'], zorder=1)
ax1.axvspan(0, 300, alpha=0.15, color=COLORS['above_fair'], zorder=1)

# Customize axes and labels
ax1.set_xlabel('Deviation from Fair Share (%)', fontsize=12, fontweight='medium',
               labelpad=10, color=COLORS['text'])
ax1.set_ylabel('Number of States', fontsize=12, fontweight='medium',
               labelpad=10, color=COLORS['text'])
ax1.set_title('THE FAIRNESS GAP: Most States Get Less Than Their Share',
              fontsize=12, fontweight='bold', pad=15, color=COLORS['text'])

# Add statistics in a clean annotation box
above_fair = (deviations > 0).sum()
below_fair = (deviations < 0).sum()
total_states = len(deviations)

# Calculate average deviation for below-fair states
avg_below = deviations[deviations < 0].mean()
avg_above = deviations[deviations > 0].mean()


# Customize grid and spines
ax1.grid(True, alpha=0.3, color=COLORS['grid'], linestyle='-', linewidth=0.5, zorder=1)
ax1.spines[['top', 'right']].set_visible(False)
ax1.spines[['left', 'bottom']].set_linewidth(1.5)
ax1.spines[['left', 'bottom']].set_color(COLORS['text'])

# ----------------------------------------------------------------------
# RIGHT PLOT: Population vs Fairness
# ----------------------------------------------------------------------
# NEW: Color points based on deviation from fair, not politics
# Use a color gradient: blue for above fair, light blue for below fair
point_colors = [COLORS['above_fair'] if d > 0 else COLORS['below_fair']
                for d in final_data['deviation']]

# Create scatter plot
scatter = ax2.scatter(final_data['population']/1e6, final_data['deviation'],
                      c=point_colors, s=100, alpha=0.8,
                      edgecolor='white', linewidth=1.5, zorder=3)

# Add horizontal line at fair point
```

```
ax2.axhline(y=0, color=COLORS['fair_line'], linestyle='-', linewidth=3,
            alpha=0.9, zorder=4, label='Fair Allocation Line')

# Label outliers based on deviation magnitude
for _, row in final_data.iterrows():
    if abs(row['deviation']) > 150:  # Only label VERY extreme outliers
        ax2.annotate(row['state_abr'],
                     (row['population']/1e6, row['deviation']),
                     xytext=(8, 8), textcoords='offset points',
                     fontsize=12, fontweight='bold', color=COLORS['outliers'])

# Add trend line
z = np.polyfit(final_data['population'], final_data['deviation'], 1)
p = np.poly1d(z)
x_range = np.array([final_data['population'].min(), final_data['population'].max()])
ax2.plot(x_range/1e6, p(x_range), color=COLORS['trend'], linewidth=3,
         alpha=0.8, label='Trend Line', zorder=2)

# Calculate and show correlation
correlation = final_data['population'].corr(final_data['deviation'])
trend_text = f'Correlation: {correlation:.2f}\nSmaller states → Higher funding'
ax2.text(0.05, 0.95, trend_text, transform=ax2.transAxes, fontsize=12.5,
         verticalalignment='top', fontweight='medium', color=COLORS['text'],
         bbox=dict(boxstyle='round,pad=0.5', facecolor='white',
                   edgecolor='#BDC3C7', alpha=0.9))

# Customize axes and labels
ax2.set_xlabel('State Population (Millions)', fontsize=12, fontweight='medium',
               labelpad=10, color=COLORS['text'])
ax2.set_ylabel('Deviation from Fair Share (%)', fontsize=12, fontweight='medium',
               labelpad=10, color=COLORS['text'])
ax2.set_title('POPULATION PENALTY: Smaller States Get More Per Person',
              fontsize=12, fontweight='bold', pad=15, color=COLORS['text'])

# NEW LEGEND: Focus on fairness, not politics
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor=COLORS['above_fair'], edgecolor='white', label='Above Fair Share'),
    Patch(facecolor=COLORS['below_fair'], edgecolor='white', label='Below Fair Share'),
    Patch(facecolor='none', edgecolor=COLORS['trend'], linewidth=2, label='Population Trend')
]
ax2.legend(handles=legend_elements, loc='upper right', framealpha=0.95)
```

```
# Customize grid and spines
ax2.grid(True, alpha=0.3, color=COLORS['grid'], linestyle='-', linewidth=0.5, zorder=1)
ax2.spines[['top', 'right']].set_visible(False)
ax2.spines[['left', 'bottom']].set_linewidth(1.5)
ax2.spines[['left', 'bottom']].set_color(COLORS['text'])

# ------------------------------------------------------------------------
# MAIN TITLE
# ------------------------------------------------------------------------
plt.suptitle('INFRASTRUCTURE FUNDING: THE POPULATION PARADOX',
             fontsize=18, fontweight='bold', color=COLORS['text'], y=1.05)

# Add subtitle
plt.figtext(0.5, 0.99, 'Small, less populous states receive disproportionately more funding p
            ha='center', fontsize=13, style='italic', color='#7F8C8D')

plt.tight_layout(pad=3)

# Add footnote
plt.figtext(0.3, -0.01, 'Note: "Fair share" = (total funding ÷ total population) × state popu
            ha='center', fontsize=12, color='#080b0c')

plt.show()
#updated fair share without polotics
```



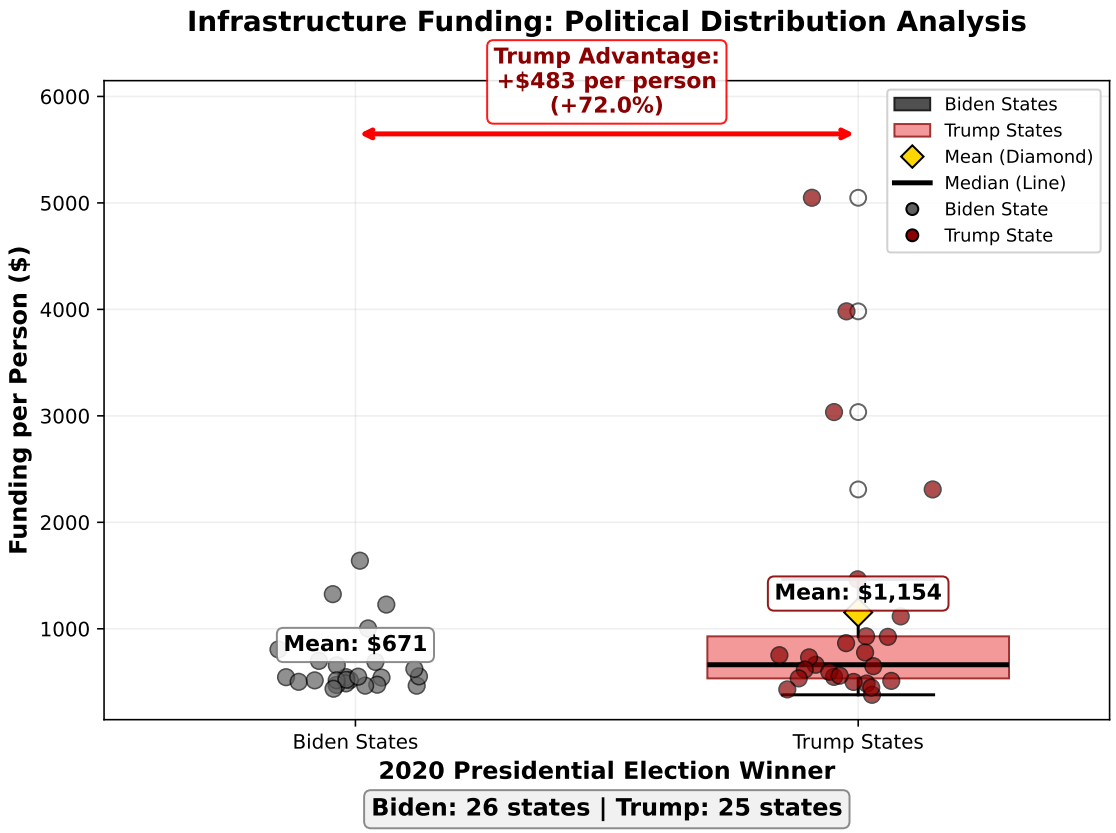
Box plot to compare the distribution of funding per capita for Biden vs Trump states, with clear styling and annotations to highlight the differences.

```
fig, ax = plt.subplots(figsize=(8, 6))
# First, create the subsets
biden_states = final_data[final_data['winner'] == 'Biden']
trump_states = final_data[final_data['winner'] == 'Trump']

# Prepare data for box plot
box_data = [biden_states['funding_per_capita'].values,
            trump_states['funding_per_capita'].values]

# Create labels
labels = ['Biden States', 'Trump States']

# Calculate statistics for annotations
biden_median = biden_states['funding_per_capita'].median()
trump_median = trump_states['funding_per_capita'].median()
biden_mean = biden_states['funding_per_capita'].mean()
trump_mean = trump_states['funding_per_capita'].mean()
# Create box plot
box_plot = ax.boxplot(box_data,
                      labels=['Biden States', 'Trump States'],
                      patch_artist=True,
                      widths=0.6,
                      medianprops=dict(color='black', linewidth=2.5),
                      whiskerprops=dict(linewidth=1.5),
                      capprops=dict(linewidth=1.5),
                      flierprops=dict(marker='o', markersize=8, alpha=0.6),
                      showmeans=True,
                      meanprops=dict(marker='D', markeredgecolor='black',
                                     markerfacecolor='gold', markersize=10))

# Color boxes - Biden in MEDIUM GREY, Trump in red
box_plot['boxes'][0].set_facecolor('lightgray')  # Medium grey
box_plot['boxes'][0].set_edgecolor('darkgray')  # Darker grey
box_plot['boxes'][0].set_alpha(1.0)
box_plot['boxes'][0].set_linewidth(2)

box_plot['boxes'][1].set_facecolor('lightcoral')
box_plot['boxes'][1].set_edgecolor('darkred')
box_plot['boxes'][1].set_alpha(0.8)
```

```
# Add individual data points with jitter
np.random.seed(42)
jitter_biden = np.random.normal(1, 0.08, size=len(biden_states))
jitter_trump = np.random.normal(2, 0.08, size=len(trump_states))

ax.scatter(jitter_biden, biden_states['funding_per_capita'],
           s=70, alpha=0.7, color='#606060', edgecolor='black', linewidth=0.8,  # Darker grey
           zorder=3, label='Individual Biden States')
ax.scatter(jitter_trump, trump_states['funding_per_capita'],
           s=70, alpha=0.7, color='darkred', edgecolor='black', linewidth=0.8,
           zorder=3, label='Individual Trump States')

# Add mean value labels - position them better
ax.text(1, biden_avg + 120, f'Mean: ${biden_avg:,.0f}',
        ha='center', fontsize=11, fontweight='bold',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9, edgecolor='#808080'))
ax.text(2, trump_avg + 120, f'Mean: ${trump_avg:,.0f}',
        ha='center', fontsize=11, fontweight='bold',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9, edgecolor='darkred'))
# Calculate the difference and percentage
difference = trump_mean - biden_mean
percent_diff = ((trump_mean / biden_mean) - 1) * 100

# Calculate arrow position (adjust based on your data range)
arrow_y = max(trump_mean, biden_mean) * 1.05  # 5% above the highest bar
# Add difference visualization - MOVED LOWER to avoid title
max_val = max(biden_states['funding_per_capita'].max(),
              trump_states['funding_per_capita'].max())
arrow_y = max_val + 600  # Increased from 400 to 600
ax.annotate('', xy=(2, arrow_y), xytext=(1, arrow_y),
            arrowprops=dict(arrowstyle='<->', color='red', lw=2.5))
ax.text(1.5, arrow_y + 200, f'Trump Advantage:\n+${difference:,.0f} per person\n(+{percent_di
        ha='center', fontsize=11, fontweight='bold', color='darkred',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9,
                  edgecolor='red', linewidth=1))

# Add sample size below x-axis
ax.text(0.5, -0.15, f'Biden: {len(biden_states)} states | Trump: {len(trump_states)} states'
        transform=ax.transAxes, ha='center', fontsize=12, fontweight='bold',
        bbox=dict(boxstyle='round', facecolor='#F0F0F0', alpha=0.9, edgecolor='#808080'))

ax.set_ylabel('Funding per Person ($)', fontsize=12, fontweight='bold')
```

```
ax.set_xlabel('2020 Presidential Election Winner', fontsize=12, fontweight='bold')
ax.set_title('Infrastructure Funding: Political Distribution Analysis',
             fontsize=14, fontweight='bold', pad=25)  # Increased padding

# Add background grid for entire plot (both x and y)
ax.grid(True, alpha=0.2, linestyle='-', which='both')  # Changed to solid lines

# Set y-axis limits to accommodate new arrow position
current_ylim = ax.get_ylim()
ax.set_ylim(current_ylim[0], arrow_y + 500)  # Extend y-axis for arrow

# Create custom legend
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#505050', edgecolor='#202020', alpha=1.0, label='Biden States'),
    Patch(facecolor='lightcoral', edgecolor='darkred', alpha=0.8, label='Trump States'),
    plt.Line2D([0], [0], marker='D', color='w', markerfacecolor='gold',
               markeredgecolor='black', markersize=8, label='Mean (Diamond)'),
    plt.Line2D([0], [0], color='black', linewidth=2.5, label='Median (Line)'),
    plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='#606060',
               markeredgecolor='black', markersize=6, label='Biden State'),
    plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='darkred',
               markeredgecolor='black', markersize=6, label='Trump State')
]
ax.legend(handles=legend_elements, loc='upper right', fontsize=9,
          framealpha=0.9, frameon=True)

plt.tight_layout()
plt.show()
```

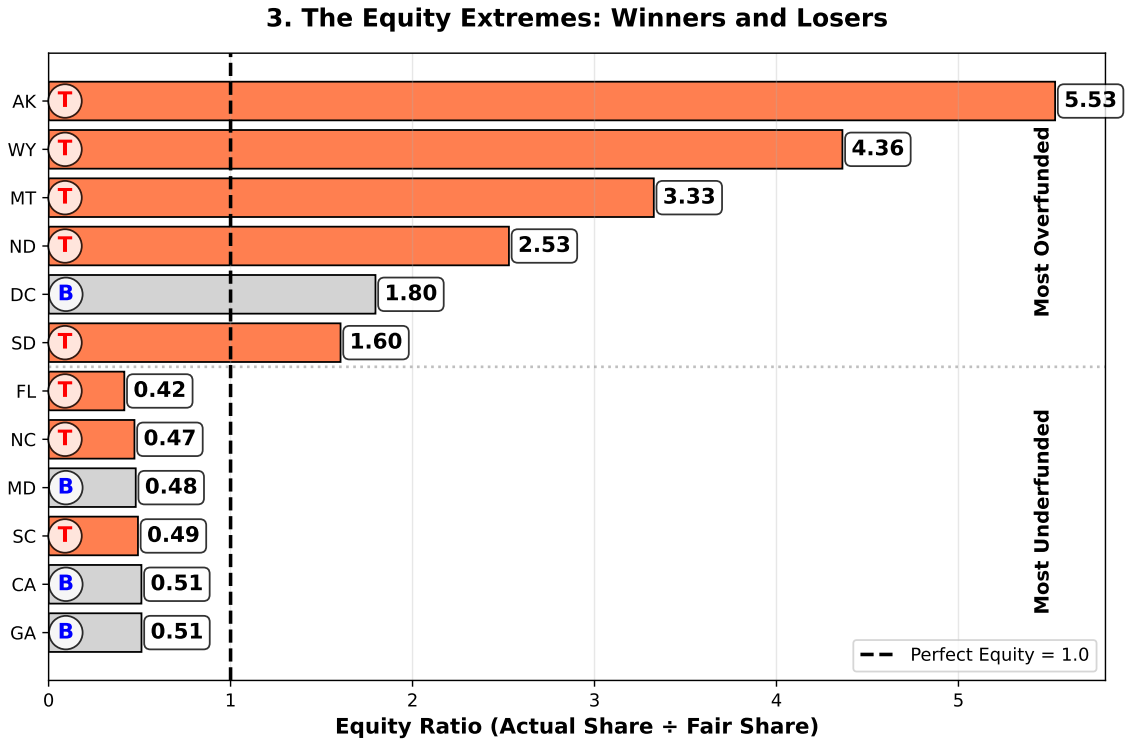
```
/var/folders/6w/rkqvbjtx04s93768hcryddtc0000gn/T/ipykernel_10445/4047633197.py:19: Matplotli
  box_plot = ax.boxplot(box_data,
```



Visualization that directly compares the equity ratio (actual funding per capita divided by the fair share) for the most overfunded and underfunded states, with clear indicators of which party won each state.

```
final_data['equity_ratio'] = final_data['funding_per_capita'] / final_data['funding_per_capi
fig, ax = plt.subplots(figsize=(9, 6))

# Get top and bottom 6 for better visibility
top_equitable = final_data.nlargest(6, 'equity_ratio')
bottom_equitable = final_data.nsmallest(6, 'equity_ratio')
equitable_comparison = pd.concat([top_equitable, bottom_equitable])

colors_equity = ['coral' if w == 'Trump' else 'lightgray' for w in equitable_comparison['win
bars = ax.barh(equitable_comparison['state_abr'], equitable_comparison['equity_ratio'],
               color=colors_equity, edgecolor='black', linewidth=1)
```

```
# Add party indicator
for i, (bar, row) in enumerate(zip(bars, equitable_comparison.iterrows())):
    row = row[1]
    party_color = 'red' if row['winner'] == 'Trump' else 'blue'
    ax.text(0.05, bar.get_y() + bar.get_height()/2, row['winner'][0],
            va='center', fontweight='bold', fontsize=12, color=party_color,
            bbox=dict(boxstyle='circle', facecolor='white', alpha=0.8))

# Add equity ratio values
for bar, ratio in zip(bars, equitable_comparison['equity_ratio']):
    width = bar.get_width()
    ax.text(width + 0.05, bar.get_y() + bar.get_height()/2,
            f'{ratio:.2f}', va='center', fontweight='bold', fontsize=12,
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

ax.axvline(x=1.0, color='black', linestyle='--', linewidth=2,
           label='Perfect Equity = 1.0')

# Add dividing line between top and bottom
mid_point = len(top_equitable) - 0.5
ax.axhline(y=mid_point, color='gray', linestyle=':', alpha=0.5)

ax.set_xlabel('Equity Ratio (Actual Share ÷ Fair Share)', fontsize=12, fontweight='bold')
ax.set_title('3. The Equity Extremes: Winners and Losers',
             fontsize=14, fontweight='bold', pad=15)
ax.invert_yaxis()
ax.grid(axis='x', alpha=0.3)
ax.legend(loc='lower right')

# Add labels for sections
ax.text(0.95, len(top_equitable)/2 - 0.5, 'Most Overfunded',
        transform=ax.get_yaxis_transform(), ha='right', va='center',
        fontweight='bold', fontsize=11, rotation=90)
ax.text(0.95, len(top_equitable) + len(bottom_equitable)/2 - 0.5, 'Most Underfunded',
        transform=ax.get_yaxis_transform(), ha='right', va='center',
        fontweight='bold', fontsize=11, rotation=90)

plt.tight_layout()
plt.show()
```



This visualization highlights the states that are most overfunded and underfunded relative to their population, with clear indicators of which party won each state in the 2020 election. The equity ratio provides a direct measure of how much more or less funding per capita each state receives compared to the national average, making it easy to identify disparities at a glance.

Visualization: A clean, infographic-style summary of the overall equity analysis, with a clear verdict and supporting evidence.

```
fig, ax = plt.subplots(figsize=(12, 9))

# Create modern color scheme
primary_colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']  # Coral, Teal, Blue
secondary_colors = ['#FFEAA7', '#A29BFE', '#FD79A8']  # Yellow, Purple, Pink
neutral_colors = ['#F5F5F5', '#E0E0E0', '#BDBDBD']  # Light grays

# Clear axis for text layout
```

```
ax.axis('off')

# Main title - moved higher
ax.text(0.5, 0.95, 'EQUITY ANALYSIS: Is Funding Proportional to Population?',
        fontsize=13, fontweight='bold', ha='center', va='top',
        color='#2D3436',
        bbox=dict(boxstyle='round', facecolor='#FFEAA7', alpha=0.9,
                  edgecolor='#FDCB6E', linewidth=2))

# Verdict box - moved down
verdict_text = f"""
VERDICT:   NOT EQUITABLE

EVIDENCE:
• Negative correlation: r = {correlation:.3f}
  (Smaller states get MORE per person)

• Only {len(final_data[final_data['equity_ratio'] >= 0.9])} states
  receive fair share (±10%)

• Extreme disparity: {final_data[final_data['state'] == 'Alaska']['funding_per_capita'].iloc
  difference between highest & lowest
"""

ax.text(0.03, 0.7, verdict_text, fontsize=11, fontweight='bold',
        linespacing=1.8, va='top', color='#2D3436',
        bbox=dict(boxstyle='round', facecolor='#F5F5F5', alpha=0.9,
                  edgecolor='#BDBDBD', linewidth=1.5))

# Right side: Visual evidence
# Small vs Large comparison
small_large_ax = ax.inset_axes([0.6, 0.45, 0.35, 0.25])
small_avg = final_data[final_data['population'] < final_data['population'].median()]['fundin
large_avg = final_data[final_data['population'] >= final_data['population'].median()]['fundi

x_pos = [0, 1]
bars = small_large_ax.bar(x_pos, [small_avg, large_avg],
                          color=[primary_colors[0], neutral_colors[1]],
                          edgecolor=['#FF3838', '#7F8C8D'], linewidth=2,
                          width=0.5)

small_large_ax.set_xticks(x_pos)
```

```
small_large_ax.set_xticklabels(['SMALL\nSTATES', 'LARGE\nSTATES'],
                               fontsize=11, fontweight='bold')
small_large_ax.set_ylabel('$ per person', fontsize=11, fontweight='bold')
small_large_ax.set_title('Per Capita Funding: Size Matters',
                         fontsize=12, fontweight='bold', pad=15,
                         color='#2D3436')
small_large_ax.tick_params(axis='y', labelsize=10)
small_large_ax.grid(axis='y', alpha=0.3, linestyle=':')

# Add stylish value labels
for i, (bar, val) in enumerate(zip(bars, [small_avg, large_avg])):
    small_large_ax.text(i, val + 100, f'${val:,.0f}',
                        ha='center', fontsize=11, fontweight='bold',
                        color=primary_colors[0] if i == 0 else '#2D3436',
                        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9,
                                  edgecolor=primary_colors[0] if i == 0 else neutral_colors[1]))

# Add arrow with difference
diff = small_avg - large_avg
if diff > 0:
    y_pos = max(small_avg, large_avg) + 400
    small_large_ax.annotate('', xy=(0.9, y_pos), xytext=(0.1, y_pos),
                            arrowprops=dict(arrowstyle='<->', color=primary_colors[0],
                                            lw=2, shrinkA=0, shrinkB=0))
    small_large_ax.text(0.5, y_pos + 150,
                        f'Small states get\n${diff:,.0f} MORE per person',
                        ha='center', fontsize=12, fontweight='bold',
                        color=primary_colors[0],
                        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9,
                                  edgecolor=primary_colors[0]))
overfunded = final_data[final_data['equity_ratio'] > 1.1]
underfunded = final_data[final_data['equity_ratio'] < 0.9]

# SIMPLE, WORKING donut chart
donut_ax = ax.inset_axes([0.6, 0.09, 0.35, 0.25])

pie_labels = ['OVERFUNDED', 'FAIR', 'UNDERFUNDED']
pie_sizes = [len(overfunded[overfunded['equity_ratio'] > 1.1]),
             len(final_data[(final_data['equity_ratio'] >= 0.9) & (final_data['equity_ratio'
             len(underfunded[underfunded['equity_ratio'] < 0.9])]
pie_colors = [primary_colors[0], primary_colors[1], neutral_colors[1]]
```

```
# Create pie chart - SIMPLEST version
wedges, texts = donut_ax.pie(pie_sizes, colors=pie_colors, startangle=90)

# Make it a donut by adding a white circle in the middle
centre_circle = plt.Circle((0, 0), 0.40, fc='white')
donut_ax.add_artist(centre_circle)

# Add percentage labels manually
total = sum(pie_sizes)
for i, (wedge, size) in enumerate(zip(wedges, pie_sizes)):
    # Calculate angle for label position
    ang = (wedge.theta2 - wedge.theta1)/2. + wedge.theta1
    y = np.sin(np.deg2rad(ang)) * 0.65
    x = np.cos(np.deg2rad(ang)) * 0.65

    percentage = (size/total)*100
    donut_ax.text(x, y, f'{percentage:.0f}%',
                  ha='center', va='center', fontsize=11, fontweight='bold',
                  color='white' if i != 2 else '#2D3436')

# Add legend with count
legend_labels = [f'{label}\n({size} states)' for label, size in zip(pie_labels, pie_sizes)]
donut_ax.legend(wedges, legend_labels, title="Equity Categories",
               loc="center left", bbox_to_anchor=(1.1, 0, 0.5, 1),
               fontsize=12, title_fontsize=11)

donut_ax.set_title('Distribution of Fairness',
                   fontsize=12, fontweight='bold', y=0.95,
                   color='#2D3436')


# Add key takeaway at bottom
ax.text(0.5, 0.02,
        'CONCLUSION: Infrastructure funding favors small states, violating population-based e
        ha='center', fontsize=12, fontweight='bold', style='italic',
        color='#2D3436',
        bbox=dict(boxstyle='round', facecolor=primary_colors[2], alpha=0.2,
                  edgecolor=primary_colors[2], linewidth=1))

plt.tight_layout(pad=3)
plt.show()
```

```
/var/folders/6w/rkqvbjtx04s93768hcryddtc0000gn/T/ipykernel_10445/2673363741.py:132: UserWarni
  plt.tight_layout(pad=3)
/Users/mehreen.gillaniicloud.com/Library/Python/3.9/lib/python/site-packages/IPython/core/pyl
  fig.canvas.print_figure(bytes_io, **kw)
```